

TP : Système de Détection de Piétons

Utilisant l'Apprentissage Profond et la Vision par Ordinateur Classique

Maryam Boufous

2 février 2026

Prérequis : C++, Bases d'OpenCV, Bases de Python

Objectifs Pédagogiques

À la fin de ce travail pratique, vous serez capable de :

- Comprendre l'architecture d'un système de détection de piétons
- Implémenter le modèle d'apprentissage profond YOLO pour la détection d'objets
- Appliquer des techniques classiques de vision par ordinateur (Canny, Transformée de Hough)
- Combiner l'apprentissage profond avec la vision classique pour une détection robuste
- Traiter des images et enregistrer les résultats de manière programmatique

Table des matières

1	Introduction	4
1.1	Qu'est-ce que la Détection de Piétons ?	4
1.2	Vue d'Ensemble du Système	4
2	Contexte Théorique	4
2.1	YOLO (You Only Look Once)	4
2.1.1	Comment Fonctionne YOLO	5
2.2	Détection de Contours de Canny	5
2.3	Transformée de Hough pour Lignes	5
2.4	Suppression Non-Maximale (NMS)	5
3	Prérequis et Installation	6
3.1	Logiciels Requis	6
3.2	Instructions d'Installation	6
3.2.1	Ubuntu/Debian	6
3.2.2	macOS	6
3.3	Télécharger les Fichiers du Modèle YOLO	6
3.4	Vérifier l'Installation	6
4	Structure du Code et Implémentation	7
4.1	Structure du Projet	7
4.2	Code Source Complet	7
4.2.1	En-têtes et Configuration	7
4.2.2	Fonction de Détection de Contours de Canny	7
4.2.3	Fonction de Détection de Structure Verticale	8
4.2.4	Fonction Principale - Partie 1 : Initialisation	10
4.2.5	Fonction Principale - Partie 2 : Chargement de YOLO	10
4.2.6	Fonction Principale - Partie 3 : Inférence	11
4.2.7	Fonction Principale - Partie 4 : Post-Traitement	12
4.2.8	Fonction Principale - Partie 5 : NMS et Visualisation	12
4.2.9	Fonction Principale - Partie 6 : Étiquettes et Sortie	14
5	Compilation et Exécution	16
5.1	Compilation	16
5.1.1	Méthode 1 : Compilation Directe	16
5.1.2	Méthode 2 : Utilisation de CMake	16
5.2	Exécution	16
6	Exercices et Défis	17
6.1	Exercice 1 : Réglage des Paramètres	17
6.2	Exercice 2 : Ajouter la Détection de Cercles pour les Têtes	17
6.3	Exercice 3 : Métriques de Performance	17
6.4	Exercice 4 : Traitement par Lot	17

7	Sujets Avancés	19
7.1	Sobel vs Prewitt vs Canny	19
7.2	Variantes de YOLO	19
7.3	Accélération GPU	19
8	Dépannage	20
8.1	Erreurs Courantes	20
8.2	Problèmes de Performance	20
9	Références et Ressources	21
9.1	Articles Académiques	21
9.2	Ressources en Ligne	21
9.3	Lectures Complémentaires	21
10	Annexe	22
10.1	Annexe A : IDs de Classe COCO	22
10.2	Annexe B : Codes Couleur OpenCV	22
10.3	Annexe C : Commandes de Référence Rapide	22

1 Introduction

1.1 Qu'est-ce que la Détection de Piétons ?

La détection de piétons est une tâche critique en vision par ordinateur qui consiste à identifier et localiser automatiquement des personnes dans des images ou des flux vidéo. Elle a de nombreuses applications concrètes :

- **Véhicules Autonomes** : Détecter les piétons pour éviter les accidents
- **Systèmes de Surveillance** : Surveiller les espaces publics pour la sécurité
- **Robotique** : Permettre aux robots de naviguer en toute sécurité autour des personnes
- **Analyse du Trafic** : Compter et suivre les piétons dans les zones urbaines

1.2 Vue d'Ensemble du Système

Notre système combine deux approches :

1. **Apprentissage Profond (YOLOv4-tiny)** : Détection rapide et précise des piétons
2. **Vision par Ordinateur Classique** : Vérification par détection de contours et de lignes

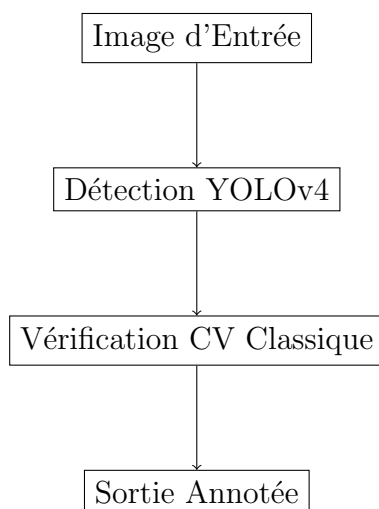


FIGURE 1 – Pipeline du Système

2 Contexte Théorique

2.1 YOLO (You Only Look Once)

YOLO est un algorithme de détection d'objets en temps réel de pointe. Contrairement aux méthodes traditionnelles qui appliquent un classificateur à plusieurs régions, YOLO :

- Traite la détection d'objets comme un **problème de régression**
- Traite l'image entière en une **seule passe avant**
- Prédit les boîtes englobantes et les probabilités de classe **simultanément**

— Atteint des performances en temps réel (30+ FPS)

YOLOv4-tiny est une version légère optimisée pour la vitesse, idéale pour les systèmes embarqués et les applications temps réel.

2.1.1 Comment Fonctionne YOLO

1. **Division en Grille** : L'image est divisée en une grille $S \times S$
2. **Prédiction de Boîtes Englobantes** : Chaque cellule de la grille prédit B boîtes englobantes
3. **Scores de Confiance** : Chaque boîte a une confiance : $P(\text{Objet}) \times \text{IoU}$
4. **Probabilités de Classe** : Chaque cellule prédit des probabilités de classe
5. **Suppression Non-Maximale** : Supprime les détections qui se chevauchent

2.2 Détection de Contours de Canny

Développé par John Canny en 1986, cet algorithme détecte les contours en plusieurs étapes :

1. **Réduction du Bruit** : Lissage par filtre gaussien
2. **Calcul du Gradient** : Trouve la force et la direction des contours
3. **Suppression Non-Maximale** : Affine les contours à une largeur d'un pixel
4. **Seuillage par Hystérésis** : Deux seuils (haut et bas) pour identifier les contours forts et faibles

Formulation Mathématique :

Magnitude du gradient : $G = \sqrt{G_x^2 + G_y^2}$

Direction du gradient : $\theta = \arctan\left(\frac{G_y}{G_x}\right)$

2.3 Transformée de Hough pour Lignes

La Transformée de Hough détecte des formes géométriques (lignes, cercles) dans les images.

Concept Clé : Une ligne dans l'espace image (x, y) est un point dans l'espace de Hough (ρ, θ)

Équation de ligne : $\rho = x \cos \theta + y \sin \theta$

Transformée de Hough Probabiliste (utilisée dans notre code) est plus rapide et retourne directement les points d'extrémité des segments de ligne.

2.4 Suppression Non-Maximale (NMS)

NMS supprime les détections en double en :

1. Triant les boîtes par score de confiance
2. Gardant la boîte avec la confiance la plus élevée
3. Supprimant les boîtes avec $\text{IoU} > \text{seuil}$ avec la boîte gardée
4. Répétant jusqu'à ce que toutes les boîtes soient traitées

IoU (Intersection over Union) :

$$\text{IoU} = \frac{\text{Aire de Chevauchement}}{\text{Aire d'Union}}$$

3 Prérequis et Installation

3.1 Logiciels Requis

1. **Compilateur C++** : g++ (version 7.0+)
2. **OpenCV** : Version 4.x avec module DNN
3. **CMake** : Version 3.10+ (optionnel mais recommandé)

3.2 Instructions d'Installation

3.2.1 Ubuntu/Debian

```
1 sudo apt-get update
2 sudo apt-get install build-essential cmake
3 sudo apt-get install libopencv-dev
```

3.2.2 macOS

```
1 brew install cmake
2 brew install opencv
```

3.3 Télécharger les Fichiers du Modèle YOLO

Téléchargez les fichiers suivants et placez-les dans le répertoire de votre projet :

1. yolov4-tiny.cfg - Configuration du modèle
2. yolov4-tiny.weights - Poids pré-entraînés

Liens de téléchargement :

```
1 # Fichier de configuration
2 wget https://raw.githubusercontent.com/AlexeyAB/darknet/master/cfg/
  yolov4-tiny.cfg
3
4 # Fichier de poids (23 MB)
5 wget https://github.com/AlexeyAB/darknet/releases/download/
  darknet_yolo_v4_pre/yolov4-tiny.weights
```

3.4 Vérifier l'Installation

Tester l'installation d'OpenCV :

```
1 pkg-config --modversion opencv4
```

4 Structure du Code et Implémentation

4.1 Structure du Projet

```
detection_pietons/  
  detecteur_pietons.cpp      # Code source principal  
  yolov4-tiny.cfg           # Configuration YOLO  
  yolov4-tiny.weights       # Poids YOLO  
  entree.jpg                # Image de test  
  sortie_detectee.jpg       # Résultat (généré)
```

4.2 Code Source Complet

4.2.1 En-têtes et Configuration

```
1 #include <opencv2/opencv.hpp>  
2 #include <opencv2/dnn.hpp>  
3 #include <iostream>  
4 #include <vector>  
5 #include <string>  
6  
7 using namespace cv;  
8 using namespace cv::dnn;  
9 using namespace std;  
10  
11 // Constantes de Configuration  
12 const string MODELE_CFG      = "yolov4-tiny.cfg";  
13 const string MODELE_POIDS   = "yolov4-tiny.weights";  
14  
15 const float SEUIL_CONF      = 0.35f; // Confiance minimale  
16 const float SEUIL_NMS      = 0.40f; // IoU NMS  
17  
18 const int LARGEUR_ENTREE    = 416; // Taille d'entr e YOLO  
19 const int HAUTEUR_ENTREE    = 416;
```

Explication :

- `opencv2/dnn.hpp` : Module de Réseau de Neurones Profonds pour YOLO
- `SEUIL_CONF` : Les détections en dessous de 35% de confiance sont rejetées
- `SEUIL_NMS` : Les boîtes avec IoU > 40% sont considérées comme des doublons
- `LARGEUR_ENTREE/HAUTEUR_ENTREE` : YOLOv4-tiny utilise une entrée de 416x416

4.2.2 Fonction de Détection de Contours de Canny

```
1 Mat obtenirContoursCanny(const Mat& roi) {  
2     if (roi.empty()) return Mat();  
3  
4     Mat gris, flou, contours;  
5  
6     // Convertir en niveaux de gris  
7     cvtColor(roi, gris, COLOR_BGR2GRAY);  
8  
9     // Appliquer un flou gaussien pour r duire le bruit  
10    GaussianBlur(gris, flou, Size(5, 5), 0);
```

```

11
12 // Appliquer la d t ection de contours de Canny
13 Canny(flou, contours, 60, 180);
14
15 return contours;
16 }

```

Explication :

- **Ligne 7** : Convertir l'image couleur BGR en niveaux de gris
- **Ligne 10** : Flou gaussien avec noyau 5x5 réduit le bruit
- **Ligne 13** : Détection de contours de Canny avec seuils 60 (bas) et 180 (haut)
- Ratio de seuil plus élevé (3 :1) fournit des contours plus nets

4.2.3 Fonction de Détection de Structure Verticale

```

1 bool aStructureVerticaleForte(const Mat& roi,
2                               double ratioVerticalMin = 0.30) {
3     // Obtenir la carte de contours
4     Mat contours = obtenirContoursCanny(roi);
5     if (contours.empty()) return false;
6
7     // D tecter les lignes avec la Transform e de Hough Probabiliste
8     vector<Vec4i> lignes;
9     HoughLinesP(contours, lignes, 1, CV_PI/180, 50, 35, 12);
10
11     if (lignes.empty()) return false;
12
13     // Compter les lignes verticales
14     int compteVerticales = 0;
15     for (const auto& l : lignes) {
16         int dx = abs(l[2] - l[0]); // Distance horizontale
17         int dy = abs(l[3] - l[1]); // Distance verticale
18
19         // La ligne est verticale si dy > 2.5 * dx
20         if (dy > dx * 2.5)
21             compteVerticales++;
22     }
23
24     // Calculer le ratio de lignes verticales
25     double ratioVertical =
26         static_cast<double>(compteVerticales) / lignes.size();
27
28     return ratioVertical >= ratioVerticalMin;
29 }

```

Explication :

- **Ligne 9** : Paramètres de HoughLinesP :
 - rho = 1 : Résolution en distance (pixels)
 - theta = CV_PI/180 : Résolution angulaire (1 degré)
 - seuil = 50 : Nombre minimum de votes requis
 - longueurLigneMin = 35 : Longueur minimale de ligne
 - ecartLigneMax = 12 : Écart maximum entre segments
- **Ligne 20** : Une ligne est verticale si hauteur $> 2.5 \times$ largeur

- **Ligne 28 :** Au moins 30% des lignes doivent être verticales
- **Justification :** Les piétons ont une forte structure corporelle verticale

4.2.4 Fonction Principale - Partie 1 : Initialisation

```

1 int main(int argc, char** argv) {
2     // Vérifier les arguments de ligne de commande
3     if (argc < 2) {
4         cout << "\nUtilisation :\n";
5         cout << " " << argv[0]
6             << " <image_entree.jpg> [image_sortie.jpg]\n\n";
7         cout << "Exemples :\n";
8         cout << " " << argv[0] << " entree.jpg\n";
9         cout << " " << argv[0] << " entree.jpg sortie.jpg\n\n";
10        return -1;
11    }
12
13    string cheminEntree = argv[1];
14    string cheminSortie = (argc >= 3) ? argv[2]
15                          : "sortie_detectee.jpg";
16
17    // Charger l'image d'entrée
18    cout << "Lecture de l'image : " << cheminEntree << "\n";
19    Mat img = imread(cheminEntree);
20
21    if (img.empty()) {
22        cerr << "ERREUR : Impossible de lire l'image : "
23             << cheminEntree << endl;
24        return -1;
25    }
26
27    cout << "Taille de l'image : " << img.cols
28         << "x" << img.rows << "\n";

```

Explication :

- **Lignes 3-11** : Valider les arguments de ligne de commande
- **Ligne 13-14** : Analyser les chemins d'entrée et de sortie
- **Ligne 18** : Charger l'image avec imread d'OpenCV
- **Ligne 20-24** : Gestion d'erreur pour images invalides

4.2.5 Fonction Principale - Partie 2 : Chargement de YOLO

```

1 // Charger le modèle YOLO
2 cout << "Chargement du modèle YOLOv4-tiny...\n";
3 Net reseau = readNetFromDarknet(MODELE_CFG, MODELE_POIDS);
4
5 // Définir le backend et la cible de calcul
6 reseau.setPreferableBackend(DNN_BACKEND_OPENCV);
7 reseau.setPreferableTarget(DNN_TARGET_CPU);
8
9 // Obtenir les noms des couches de sortie
10 vector<String> nomsSorties = reseau.getUnconnectedOutLayersNames();

```

Explication :

- **Ligne 3** : Charger YOLO au format Darknet (.cfg + .weights)
- **Ligne 6** : Utiliser le backend d'OpenCV (alternatives : CUDA, OpenCL)
- **Ligne 7** : Définir la cible de calcul (CPU, GPU, etc.)
- **Ligne 10** : Obtenir les noms des couches de sortie de YOLO

4.2.6 Fonction Principale - Partie 3 : Inférence

```
1 // Préparer le blob d'entrée
2 cout << "Détection des piétons...\n";
3 Mat blob = blobFromImage(
4     img, // Image d'entrée
5     1/255.0, // Facteur d'échelle (normalisation)
6     Size(LARGEUR_ENTREE, HAUTEUR_ENTREE), // Taille cible
7     Scalar(0,0,0), // Soustraction de moyenne
8     true, // changer les canaux R et B
9     false // Rogner l'image
10 );
11 reseau.setInput(blob);
12
13 // Passe avant travers le réseau
14 vector<Mat> sorties;
15 reseau.forward(sorties, nomsSorties);
```

Explication :

- **blobFromImage** : Prépare l'image pour le réseau de neurones :
 - Redimensionne à 416x416
 - Normalise les valeurs de pixels à [0, 1]
 - Échange RGB en BGR (OpenCV utilise BGR)
- **Ligne 11** : Définir le blob comme entrée du réseau
- **Ligne 15** : Exécuter l'inférence et obtenir les prédictions

4.2.7 Fonction Principale - Partie 4 : Post-Traitement

```

1  // Analyser les sorties YOLO
2  vector<int> idsClasses;
3  vector<float> confiances;
4  vector<Rect> boites;
5
6  for (size_t i = 0; i < sorties.size(); ++i) {
7      float* donnees = (float*)sorties[i].data;
8
9      for (int rangee = 0; rangee < sorties[i].rows;
10         ++rangee, donnees += sorties[i].cols) {
11
12         // Extraire les scores de classe
13         Mat scores = sorties[i].row(rangee).colRange(5,
14                                                    sorties[i].cols);
15
16         Point pointIdClasse;
17         double conf;
18         minMaxLoc(scores, nullptr, &conf, nullptr,
19                    &pointIdClasse);
20
21         // Filtrer par confiance et classe (personne = 0)
22         if (conf > SEUIL_CONF && pointIdClasse.x == 0) {
23             // Extraire les coordonnées de la boîte englobante
24             float centreX = donnees[0] * img.cols;
25             float centreY = donnees[1] * img.rows;
26             float l       = donnees[2] * img.cols;
27             float h       = donnees[3] * img.rows;
28
29             int gauche = max(0, (int)(centreX - l/2));
30             int haut   = max(0, (int)(centreY - h/2));
31
32             idsClasses.push_back(0);
33             confiances.push_back(static_cast<float>(conf));
34             boites.emplace_back(gauche, haut, (int)l, (int)h);
35         }
36     }

```

Explication :

- **Format de Sortie YOLO** : Chaque détection contient :
 - `donnees[0]`, `donnees[1]` : Coordonnées du centre (normalisées)
 - `donnees[2]`, `donnees[3]` : Largeur et hauteur (normalisées)
 - `donnees[4]` : Score d'objectivité
 - `donnees[5+]` : Probabilités de classe
- **Ligne 20** : Filtrer pour la classe "personne" (ID classe = 0 dans le jeu de données COCO)
- **Lignes 22-25** : Convertir les coordonnées normalisées en valeurs de pixels
- **Lignes 27-28** : Convertir centre+taille en coin supérieur gauche

4.2.8 Fonction Principale - Partie 5 : NMS et Visualisation

```

1  // Appliquer la Suppression Non-Maximale
2  vector<int> indices;

```

```

3      NMSBoxes(boites, confiances, SEUIL_CONF,
4              SEUIL_NMS, indices);
5
6      cout << "\n===== \n";
7      cout << "Trouv " << indices.size()
8            << " pi ton(s)\n";
9      cout << "===== \n";
10
11     // Dessiner les r sultats
12     Mat resultat = img.clone();
13
14     for (size_t i = 0; i < indices.size(); i++) {
15         int idx = indices[i];
16         Rect boite = boites[idx];
17         float conf = confiances[idx];
18
19         // S'assurer que la bo te est dans les limites de l'image
20         boite.x = max(0, boite.x);
21         boite.y = max(0, boite.y);
22         boite.width = min(boite.width, img.cols - boite.x);
23         boite.height = min(boite.height, img.rows - boite.y);
24
25         // V rifier avec CV classique
26         Mat roi = img(boite);
27         bool cv_classique_ok = aStructureVerticaleForte(roi);
28
29         // Couleur : vert = confirm , orange = faible
30         Scalar couleur = cv_classique_ok ? Scalar(0, 220, 30)
31                                     : Scalar(0, 140, 255);
32
33         // Dessiner la bo te englobante
34         rectangle(resultat, boite, couleur, 3);

```

Explication :

- **Ligne 3** : NMSBoxes supprime les détections qui se chevauchent
- **Ligne 26** : Extraire la Région d'Intérêt (ROI) pour vérification
- **Ligne 27** : Vérifier si le piéton a une structure verticale
- **Ligne 30-31** : Codage couleur :
 - Vert (0, 220, 30) : Contours forts détectés
 - Orange (0, 140, 255) : Contours faibles (YOLO uniquement)

4.2.9 Fonction Principale - Partie 6 : Étiquettes et Sortie

```

1 // Cr e r l' tiquette
2 string etiquette = "Personne " + to_string(i + 1) + " " +
3                     to_string(int(conf * 100 + 0.5)) + "%";
4 if (!cv_classique_ok)
5     etiquette += " (contours faibles)";
6
7 // Dessiner l'arri re-plan de l' tiquette
8 int ligneBase = 0;
9 Size tailleTexte = getTextSize(etiquette,
10                                FONT_HERSHEY_SIMPLEX,
11                                0.65, 2, &ligneBase);
12 Point orgTexte(boite.x, boite.y - 8);
13
14 rectangle(resultat, orgTexte + Point(0, ligneBase),
15            orgTexte + Point(tailleTexte.width,
16                              -tailleTexte.height - ligneBase),
17            couleur, FILLED);
18
19 // Dessiner le texte de l' tiquette
20 putText(resultat, etiquette, orgTexte,
21          FONT_HERSHEY_SIMPLEX, 0.65,
22          Scalar(255,255,255), 2);
23
24 // Sortie console
25 cout << "Personne " << (i+1) << " :\n";
26 cout << " Confiance : " << int(conf*100) << "%\n";
27 cout << " Position : (" << boite.x << ", "
28         << boite.y << ")\n";
29 cout << " Taille : " << boite.width << "x"
30         << boite.height << "\n";
31 cout << " V rification contours : "
32         << (cv_classique_ok ? "FORTE (confirm e)"
33                : "FAIBLE") << "\n\n";
34 }
35
36 // Enregistrer l'image de sortie
37 imwrite(cheminSortie, resultat);
38 cout << "Sortie enregistr e dans : " << cheminSortie << "\n";
39
40 // Afficher le r sultat
41 cout << "\nAffichage du r sultat. Appuyez sur une touche pour
42 quitter...\n";
43 imshow("R sultat de la D tecti on de Pi tons", resultat);
44 waitKey(0);
45 destroyAllWindows();
46
47 cout << "\nTermin e!\n";
48 return 0;
49 }

```

Explication :

- **Lignes 2-5** : Créer une étiquette montrant le numéro de personne et la confiance
- **Lignes 8-17** : Dessiner un rectangle rempli pour l'arrière-plan de l'étiquette
- **Lignes 20-22** : Dessiner le texte blanc sur l'arrière-plan
- **Lignes 25-33** : Afficher les informations détaillées de détection

- **Ligne 37** : Enregistrer l'image annotée sur le disque
- **Lignes 41-43** : Afficher le résultat et attendre une pression de touche

5 Compilation et Exécution

5.1 Compilation

5.1.1 Méthode 1 : Compilation Directe

```
1 g++ -std=c++11 detecteur_pietons.cpp -o detecteur_pietons \  
2 'pkg-config --cflags --libs opencv4'
```

5.1.2 Méthode 2 : Utilisation de CMake

Créer CMakeLists.txt :

```
1 cmake_minimum_required(VERSION 3.10)  
2 project(DetectionPietons)  
3  
4 set(CMAKE_CXX_STANDARD 11)  
5  
6 find_package(OpenCV REQUIRED)  
7 include_directories(${OpenCV_INCLUDE_DIRS})  
8  
9 add_executable(detecteur_pietons detecteur_pietons.cpp)  
10 target_link_libraries(detecteur_pietons ${OpenCV_LIBS})
```

Puis compiler :

```
1 mkdir build && cd build  
2 cmake ..  
3 make
```

5.2 Exécution

```
1 # Utilisation de base (sortie enregistrée comme sortie_detectee.jpg)  
2 ./detecteur_pietons entree.jpg  
3  
4 # Spécifier le nom du fichier de sortie  
5 ./detecteur_pietons entree.jpg resultat.jpg  
  
]
```


6 Exercices et Défis

6.1 Exercice 1 : Réglage des Paramètres

Modifiez les paramètres suivants et observez les effets :

1. Changez SEUIL_CONF de 0.35 à 0.50 et 0.20
 - **Question** : Comment cela affecte-t-il les faux positifs et les faux négatifs ?
2. Changez les seuils de Canny de (60, 180) à (50, 150) et (100, 200)
 - **Question** : Comment la sensibilité de détection de contours change-t-elle ?
3. Modifiez ratioVerticalMin de 0.30 à 0.40 et 0.20
 - **Question** : Combien de détections obtiennent des boîtes orange vs vertes ?

6.2 Exercice 2 : Ajouter la Détection de Cercles pour les Têtes

Étendez la fonction de vérification pour détecter également les formes circulaires (têtes) :

```
1 bool aStructureTete(const Mat& roi) {  
2     // TODO : Implémenter la détection de tête avec HoughCircles  
3     // Indice : Chercher des cercles dans les 30% supérieurs du ROI  
4     // Rayon attendu : 10-30% de la largeur du ROI  
5 }
```

Indices :

- Utilisez la fonction HoughCircles()
- Recherchez dans l'image en niveaux de gris
- Rayon de cercle attendu : 10-30% de la largeur de la boîte de détection
- Cherchez dans la partie supérieure de la boîte englobante

6.3 Exercice 3 : Métriques de Performance

Implémentez une fonction pour calculer les métriques de détection :

```
1 struct Metriques {  
2     int totalDetections;  
3     int detectionsContoursFortes;  
4     int detectionsContoursFaibles;  
5     double confianceMoyenne;  
6     double tempsTraitement;  
7 };  
8  
9 Metriques calculerMetriques(/* paramètres */) {  
10     // TODO : Implémenter le calcul des métriques  
11 }
```

6.4 Exercice 4 : Traitement par Lot

Modifiez le programme pour traiter plusieurs images dans un dossier :

```
1 // Traiter toutes les images d'un repertoire
2 void traiterRepertoire(const string& repEntree,
3                        const string& repSortie) {
4     // TODO : Implémenter le traitement par lot
5 }
```

Exigences :

- Lire tous les fichiers .jpg et .png du répertoire d'entrée
- Traiter chaque image
- Enregistrer les résultats dans le répertoire de sortie
- Générer un rapport récapitulatif (fichier CSV)

7 Sujets Avancés

7.1 Sobel vs Prewitt vs Canny

Comparez différentes méthodes de détection de contours :

Méthode	Qualité	Vitesse	Résistance au bruit
Canny	Excellente	Moyenne	Excellente
Sobel	Bonne	Rapide	Bonne
Prewitt	Bonne	Rapide	Correcte

TABLE 1 – Comparaison de Détection de Contours

Exercice : Remplacez Canny par Sobel et comparez les résultats.

7.2 Variantes de YOLO

Modèle	Taille	Vitesse	Précision
YOLOv4-tiny	23 MB	Très rapide	Bonne
YOLOv4	246 MB	Rapide	Excellente
YOLOv5s	14 MB	Très rapide	Bonne
YOLOv8	Variable	Rapide	Excellente

TABLE 2 – Comparaison des Modèles YOLO

7.3 Accélération GPU

Pour activer l'accélération GPU, modifiez le backend :

```

1 // Changer de :
2 reseau.setPreferableBackend(DNN_BACKEND_OPENCV);
3 reseau.setPreferableTarget(DNN_TARGET_CPU);
4
5 //      :
6 reseau.setPreferableBackend(DNN_BACKEND_CUDA);
7 reseau.setPreferableTarget(DNN_TARGET_CUDA);

```

Note : Nécessite OpenCV compilé avec support CUDA.

8 Dépannage

8.1 Erreurs Courantes

Erreur : Impossible de lire l'image

Cause : Chemin de fichier invalide ou format non supporté

Solution :

- Vérifier que le chemin du fichier est correct
- Utiliser un chemin absolu si le chemin relatif échoue
- S'assurer que le format d'image est supporté (.jpg, .png, .bmp)

Erreur : Impossible de charger le modèle YOLO

Cause : Fichiers .cfg ou .weights manquants

Solution :

- Télécharger les deux fichiers dans le répertoire du projet
- Vérifier les tailles de fichiers (weights devrait être 23 MB)
- Vérifier les permissions des fichiers

Erreur : OpenCV non trouvé pendant la compilation

Cause : OpenCV non installé ou problème pkg-config

Solution :

```
1 # Vérifier l'installation d'OpenCV
2 pkg-config --modversion opencv4
3
4 # Si non trouvé, réinstaller
5 sudo apt-get install libopencv-dev
```

8.2 Problèmes de Performance

Détection lente :

- Réduire la taille de l'image d'entrée avant traitement
- Utiliser YOLOv4-tiny au lieu de YOLOv4 complet
- Activer l'accélération GPU si disponible

Précision de détection médiocre :

- Abaisser SEUIL_CONF (plus de détections, plus de faux positifs)
- Utiliser des images d'entrée haute résolution
- Assurer un bon éclairage dans les images

9 Références et Ressources

9.1 Articles Académiques

1. Redmon, J., & Farhadi, A. (2018). YOLOv3 : An Incremental Improvement. arXiv preprint arXiv :1804.02767.
2. Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). YOLOv4 : Optimal Speed and Accuracy of Object Detection. arXiv preprint arXiv :2004.10934.
3. Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8(6), 679-698.
4. Duda, R. O., & Hart, P. E. (1972). Use of the Hough Transformation to Detect Lines and Curves in Pictures. Communications of the ACM, 15(1), 11-15.

9.2 Ressources en Ligne

- Documentation OpenCV : <https://docs.opencv.org/4.x/>
- Site Officiel YOLO : <https://pjreddie.com/darknet/yolo/>
- Tutoriel Module DNN OpenCV : https://docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html
- GitHub Darknet : <https://github.com/AlexeyAB/darknet>

9.3 Lectures Complémentaires

- Computer Vision : Algorithms and Applications par Richard Szeliski
- Deep Learning for Computer Vision par Rajalingappaa Shanmugamani
- Learning OpenCV 4 Computer Vision with Python 3 par Joseph Howse

10 Annexe

10.1 Annexe A : IDs de Classe COCO

YOLOv4 est entraîné sur le jeu de données COCO. IDs de classe courants :

ID	Classe	ID	Classe
0	personne	5	bus
1	vélo	6	train
2	voiture	7	camion
3	moto	8	bateau
4	avion	16	chien

TABLE 3 – Classes du Jeu de Données COCO (liste partielle)

10.2 Annexe B : Codes Couleur OpenCV

Format de couleur BGR (pas RGB) :

Couleur	Code BGR
Rouge	(0, 0, 255)
Vert	(0, 255, 0)
Bleu	(255, 0, 0)
Jaune	(0, 255, 255)
Orange	(0, 140, 255)
Blanc	(255, 255, 255)
Noir	(0, 0, 0)

TABLE 4 – Couleurs BGR Courantes

10.3 Annexe C : Commandes de Référence Rapide

```

1 # Installer les dépendances
2 sudo apt-get install libopencv-dev
3
4 # Télécharger les fichiers YOLO
5 wget https://raw.githubusercontent.com/AlexeyAB/darknet/master/cfg/
  yolov4-tiny.cfg
6 wget https://github.com/AlexeyAB/darknet/releases/download/
  darknet_yolo_v4_pre/yolov4-tiny.weights
7
8 # Compiler
9 g++ -std=c++11 detecteur_pietons.cpp -o detecteur_pietons `pkg-config --
  cflags --libs opencv4`
10
11 # Exécuter
12 ./detecteur_pietons entree.jpg sortie.jpg
13
14 # Vérifier la version d'OpenCV
15 pkg-config --modversion opencv4

```

Bonne Chance !